PROYECTO FINAL LENGUAJES FORMALES Y AUTOMATAS 2026 2 — INGENIERÍA DE SISTEMAS Y COMPUTACIÓN-UPTC SOGAMOSO

Validación en Tiempo Real de Esquemas JSON/XML mediante Autómatas de Pila Ponderados

1. Descripción general

Los microservicios consumen una parte considerable de CPU al serializar y parsear contratos de API: cada payload se convierte primero en un árbol genérico (AST/DOM) y luego se valida de forma interpretada contra el esquema. Los estudiantes desarrollarán un compilador que traduce un esquema JSON (JSON Schema, con extensión a un subconjunto de XSD) a un autómata de pila determinista (DPDA) ponderado (DPDA) y emite un validador nativo en C y Rust que consume los bytes del payload en una sola pasada, sin construir un AST ni reservar memoria dinámica. Se aplican los temas del curso: gramáticas y lenguajes formales, autómatas finitos y de pila, análisis léxico, sintáctico y semántico, generación de código y análisis de complejidad.

2. Objetivos

- Definir formalmente el subconjunto de JSON Schema (Draft 2020-12) cuyo lenguaje es reconocible por un DPDA con registros acotados, y clasificar el resto de palabras clave.
- Construir el prototipo de compilador de esquema → DPDA ponderado → código nativo C/Rust con validación en streaming.
- Medir en vivo rendimiento, memoria y asignaciones frente a validadores basados en AST y contrastarlos con la complejidad teórica.
- Desplegar la solución como aplicación web en la nube con persistencia NoSQL.

3. Componentes a desarrollar

- Fase 1 — Análisis léxico. Alfabeto (bytes UTF-8), delimitadores y tokens de JSON y XML; AFD del tokenizador en streaming (números, cadenas con escapes, literales, etiquetas); gramáticas regulares y diagramas de autómatas; lexer del propio esquema.
- Fase 2 — Análisis sintáctico. Gramática libre de contexto de JSON y de XML bien formado, y del metalenguaje de esquemas; parser descendente LL (1) del esquema (el AST existe solo en tiempo de compilación); árboles de derivación de payloads de ejemplo.
- Fase 3 — Análisis semántico del esquema. Validación contra el meta-esquema, resolución de $ref/$defs con tabla de símbolos y detección de ciclos no productivos, y clasificación de palabras clave: (a) directas en el DPDA (type, properties, items, enum, const, límites); (b) con registros o contadores (required mediante máscara de bits, minItems/maxItems); (c) no deterministas o no libres de contexto (anyOf/oneOf/allOf, pattern, uniqueItems, dependencias), con estrategia de producto de autómatas, determinización acotada o verificación diferida.
- Fase 4 — Construcción del autómata ponderado. Definición formal $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F, w)$ con pesos en el semianillo tropical: el peso estima el costo de cada transición para ordenar comprobaciones (fallo temprano) y acumula el número de violaciones para reportar la conformidad del payload; construcción composicional por nodo del esquema, verificación de determinismo de δ y minimización de estados.
- Fase 5 — Generación de código. Backend que emite C y Rust: tablas de transición densas o saltos directos, pila de profundidad acotada y configurable, sin asignación dinámica, API C-ABI/FFI y opcionalmente WASM; generador de payloads válidos e inválidos (serialización JSON/XML) a partir del esquema para alimentar las pruebas.
- Fase 6 — Validación en tiempo real. Modo streaming por bloques, error con ruta JSON Pointer y posición en bytes, manejo de payloads malformados y límites de profundidad, tamaño y tiempo; instrumentación en vivo de MB/s, ciclos por byte, pico de memoria, asignaciones, profundidad de pila y tamaño del autómata (estados y transiciones), con línea base de DOM + validador genérico.
- Fase 7 — Aplicación web. Backend (API REST + WebSocket) y frontend con editor de esquema, compilación y descarga del código generado, envío de payloads, visualización del autómata y del árbol de derivación, y panel de comparación en vivo. Compilación y ejecución en sandbox; despliegue en AWS o Azure.

4. Casos de prueba

Todos los casos (esquema, payload, resultado esperado y métricas) se almacenan en una base de datos NoSQL en la nube (DynamoDB, Cosmos DB o MongoDB Atlas).

- Válidos: esquemas simples, anidados y recursivos ($ref, árboles) en JSON y XML.
- Inválidos: tipos incorrectos, campos obligatorios faltantes, propiedades adicionales, violaciones de enum y rangos.
- Malformados: sintaxis incorrecta, UTF-8 inválido, claves duplicadas, etiquetas XML sin cerrar.
- Fuera del DPDA puro: anyOf/oneOf ambiguos, pattern y uniqueItems con su estrategia de respaldo.

Validación en Tiempo Real de Esquemas JSON/XML mediante Autómatas de Pila Ponderados

- Pruebas de bombeo: payloads $u \cdot v^k \cdot x \cdot y^k \cdot z$ con anidamiento creciente (k), arreglos muy largos y carga sostenida, midiendo tiempo, pila y memoria.
- Extremos: payloads de gran tamaño en streaming, profundidad sobre el límite, números enormes, expansión de entidades XML.

5. Entregables

1. Documentación técnica: definición formal del lenguaje, diagramas de autómatas (AFD, DPDA), gramáticas regulares y libres de contexto, arquitectura del compilador y árboles de derivación.
2. Código fuente: compilador y validadores generados, modulares y documentados, con suite completa de pruebas.
3. Aplicación funcional: sistema web (backend y frontend) desplegado en la nube, con los casos de prueba en base de datos NoSQL.
4. Informe final: análisis de complejidad (validación O(n) en bytes y O(d) en pila; tamaño del autómata y riesgo de explosión al determinizar), decisiones de diseño, manejo de errores y casos especiales, vulnerabilidades detectadas y mitigadas (ReDoS en pattern, desbordamiento de pila por anidamiento, XXE y expansión de entidades, $ref cíclicos, ejecución de código generado) y resultados de las pruebas de bombeo.
5. Presentación: demo en vivo, explicación de las fases y demostración de los casos de prueba con métricas comparativas.